

International Islamic University Islamabad

Faculty of Engineering and Technology

Department of Electrical and Computer Engineering



AI & Machine Learning Lab

Experiment No. 11: Advanced Classification Using Decision Tree, Random Forest, and SVM.

Name of Student: Bushra Nazir

Registration No.: 1071-F22

Date of Experiment: 27-5-2026

Submitted To: Engr.Asad

Objectives:

1. To understand and implement advanced classification algorithms including Decision Tree, Random Forest, and Support Vector Machine (SVM) using Python-based tools.
2. To compare the performance of different classification models using confusion matrix and accuracy metrics across multiple datasets.
3. To deploy a trained Decision Tree model on a Raspberry Pi for real-time classification using sensor input.

Equipment Required:

- PC/Laptop with Jupyter Notebook or Google Colab access

Introduction :

In this lab, students will explore a deeper layer of classification techniques by working with Decision Trees, Random Forests, and Support Vector Machines (SVM) three powerful yet fundamentally different machine learning models. Building upon the foundational concepts learned in the previous lab with Logistic Regression and KNN, this experiment focuses on models capable of handling more complex, non-linear relationships in data.

Students will not only implement and compare these models using practical datasets but also evaluate their effectiveness through confusion matrices and classification reports. Moreover, this lab goes a step further by deploying a trained Decision Tree classifier on a Raspberry Pi, where real-time sensor data will be used for live predictions. This blend of theoretical understanding, software modeling, and hardware deployment provides students with a comprehensive view of how machine learning can be applied in real-world embedded systems.

Decision Tree Classifier

A Decision Tree is a supervised learning algorithm used for both classification and regression tasks. In classification, it splits the data into branches based on feature values, forming a tree-like structure where each internal node represents a decision on

a feature, each branch represents an outcome of the decision, and each leaf node represents a class label.

The tree uses questions or conditions to divide the data in a recursive manner. These decisions are made using metrics like Gini Index or Information Gain (based on Entropy) to determine the best feature and threshold for splitting.

Imagine a system that classifies whether a person will play tennis based on the weather. The decision might be based on three features:

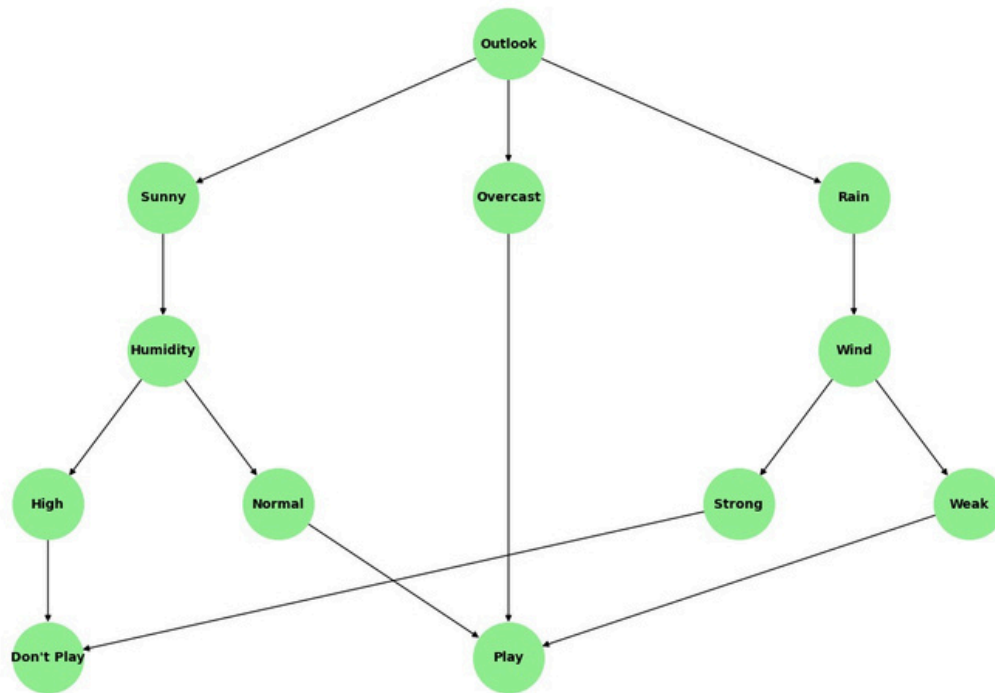
- Outlook (Sunny, Overcast, Rain)
- Humidity (High, Normal)
- Wind (Strong, Weak)

The tree would ask a sequence of questions like:

- Is the outlook sunny?
 - If yes, is humidity high?
 - If yes → Don't Play
 - If no → Play

This chain of decisions forms the decision tree.

Decision Tree: Should You Play Tennis?



Implementation using Python

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix

# Load dataset
iris = load_iris()
```

```

X =pd.DataFrame(iris.data, columns=iris.feature_names)
y =pd.Series(iris.target)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# TrainDecision Tree model
clf = DecisionTreeClassifier(criterion='entropy') #or 'gini'
clf.fit(X_train, y_train)

# Prediction and Evaluation
y_pred = clf.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
report = classification_report(y_test, y_pred,
target_names=iris.target_names, output_dict=True)

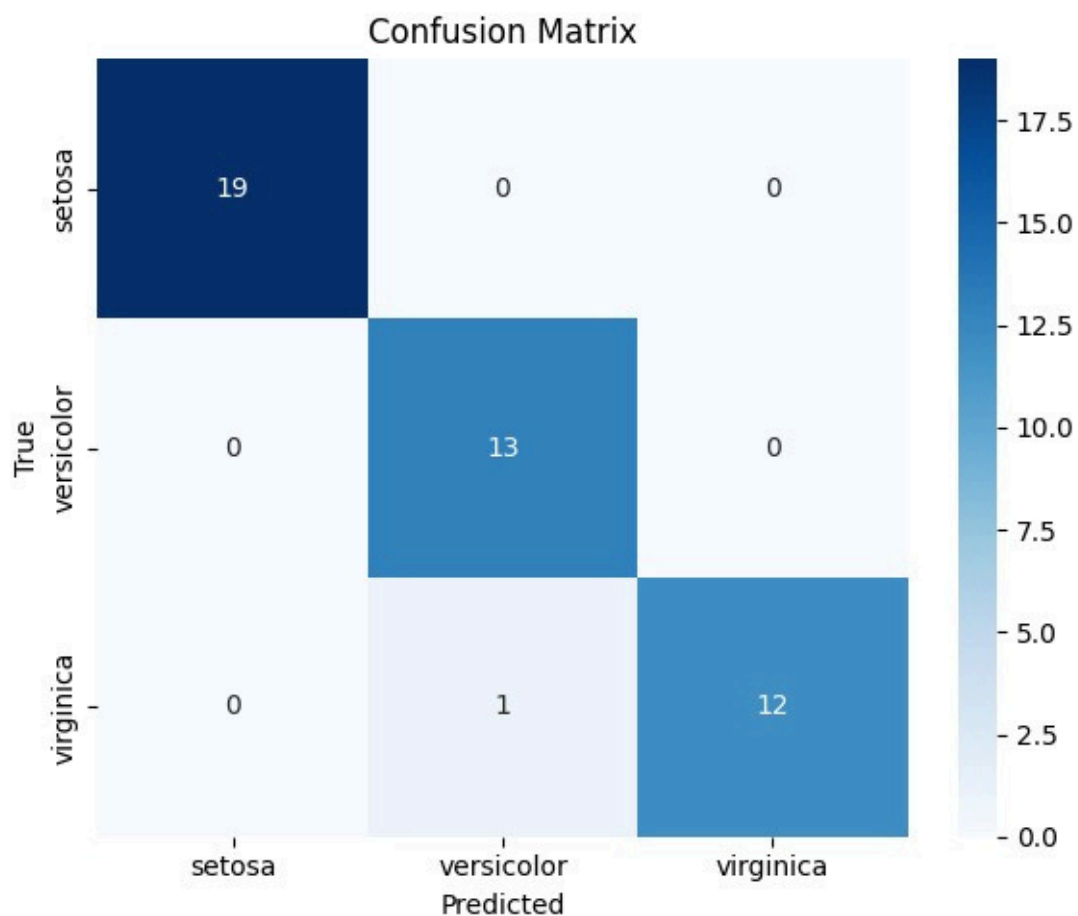
# Visualize Confusion Matrix
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix")
plt.tight_layout()
plt.show()

# Visualize Classification Report
report_df = pd.DataFrame(report).transpose()
plt.figure(figsize=(8, 4))
sns.heatmap(report_df.iloc[:1, :-1], annot=True, cmap='YlGnBu',
cbar=False)
plt.title("Classification Report")
plt.tight_layout()
plt.show()

#Visualize Decision Tree
plt.figure(figsize=(12, 8))
plot_tree(clf, filled=True, feature_names=iris.feature_names,
class_names=iris.target_names)
plt.title("Decision Tree on Iris Dataset")
plt.show()

```

Output

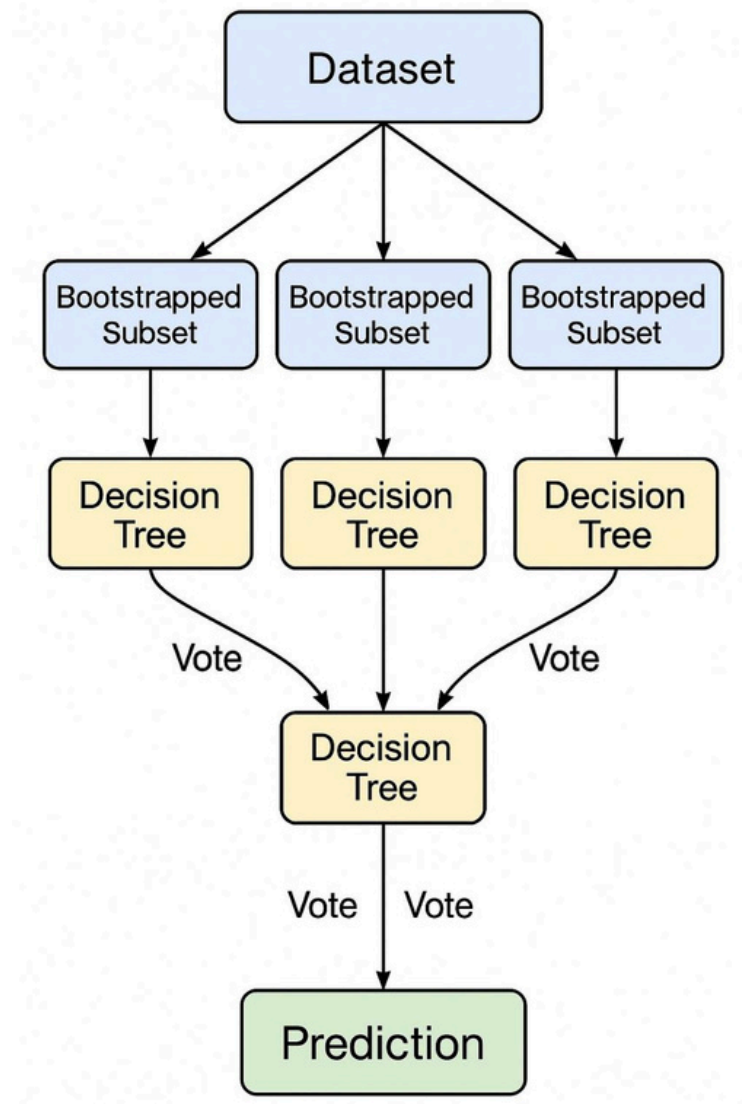


Random Forest Classifier

The Random Forest Classifier is an ensemble learning algorithm that builds multiple decision trees and merges their results to achieve better accuracy and robustness. It belongs to the class of bagging algorithms, which reduce variance by training several models on random subsets of the data and aggregating their predictions. Random Forest mitigates the major limitations of individual decision trees, such as overfitting, by averaging the outcomes of multiple trees. It is widely used in both classification and regression tasks and is known for its high accuracy and robustness to noise and outliers.

Imagine asking a question to 100 experts and taking a majority vote of their answers. That's how a Random Forest works. Instead of relying on one "expert" (decision tree), it asks many and uses majority voting (for classification) **or** averaging (for regression) to make a more reliable decision.

The working can be illustrated by following flow chart.



Pseudo code for Random Forest

Algorithm: RandomForestClassifier

Input: Dataset D, number of trees T, number of features F

Output: Class label for new data

1. For $t = 1$ to T :

- a. Draw a bootstrap sample D_t from D
- b. Train a decision tree h_t on D_t
 - At each node, select F random features
 - Choose best split among selected features

2. For prediction on input x :

- a. Let each tree h_t output a class label c_t
- b. Return the class label with majority vote among $\{c_1, \dots, c_T\}$

Implementation on python

Now we will use the Wine dataset from scikit learn library to classify different wine types based on 13 chemical features. A Random Forest Classifier is trained, and we evaluate its performance using a colored confusion matrix, classification report, and a feature importance plot.

```
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report
import matplotlib.pyplot as plt
import seaborn as sns

# Loaddataset
wine=load_wine()
X =pd.DataFrame(wine.data, columns=wine.feature_names)
y =pd.Series(wine.target)

# Splitdataset
X_train,X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Train Random Forest model
rf=RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

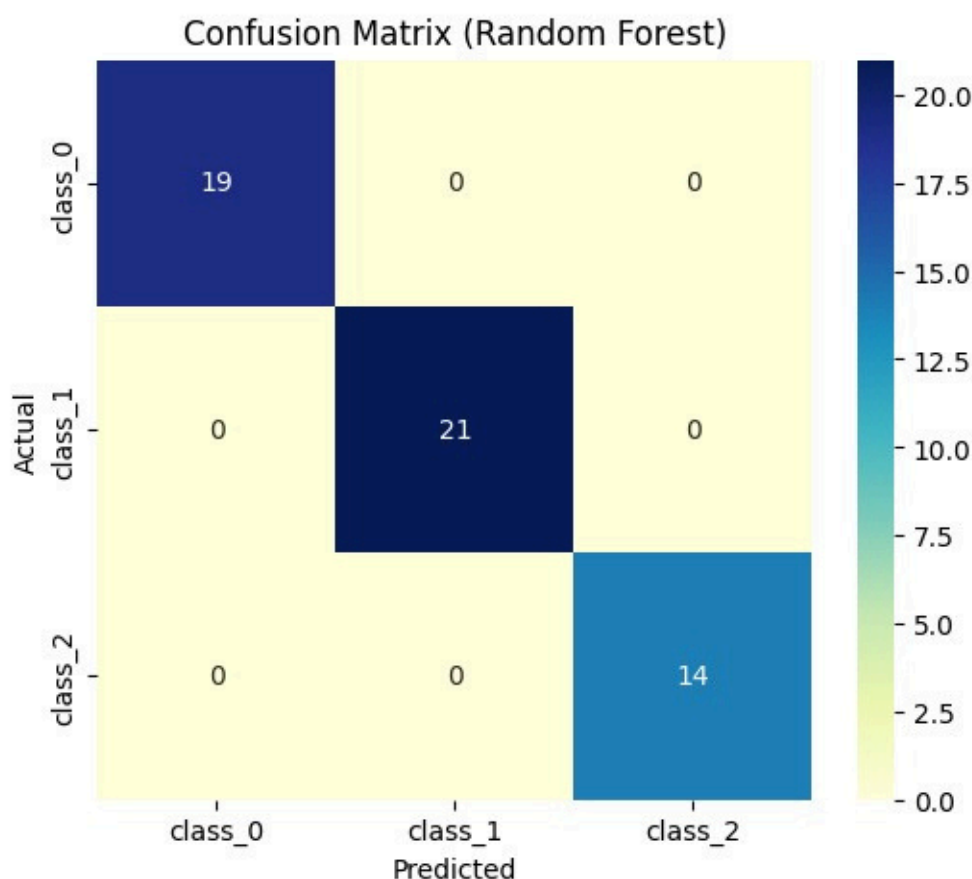
# Predictions
y_pred = rf.predict(X_test)
# Confusion Matrix (colored)
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="YlGnBu",
xticklabels=wine.target_names, yticklabels=wine.target_names)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix (Random Forest)")
plt.show()

# Classification Report
print("\nClassification Report:\n", classification_report(y_test,
y_pred, target_names=wine.target_names))
```



```
# Feature Importance Plot (Vertical)
importances = rf.feature_importances_
features = X.columns
plt.figure(figsize=(10, 8))
sns.barplot(y=features, x=importances, palette="Blues_d")
plt.title("Feature Importance in Random Forest")
plt.xlabel("Importance Score")
plt.ylabel("Features")
plt.tight_layout()
plt.show()
```

OUTPUT



The Random Forest classifier gives high accuracy and shows which chemical properties contribute most to the classification. Compared to a single Decision Tree, Random Forest reduces overfitting and improves generalization by averaging multiple trees. While a Decision Tree might misclassify on test data due to its sensitivity to splits,

Random Forest is generally more robust and accurate.

Support Vector Machine (SVM)

Support Vector Machine (SVM) is a powerful supervised learning algorithm used for both classification and regression tasks, but it is most commonly applied to binary and multiclass classification problems. The core idea of SVM is to find the optimal hyperplane that best separates the data into distinct classes with the maximum margin the distance between the hyperplane and the closest data points from each class, known as support vectors.

Unlike decision trees and random forests, which work by splitting data recursively, SVM works in a geometric space, making it especially effective in high-dimensional datasets. With the use of kernel functions, SVMs can also efficiently perform non-linear classification. Some key concepts in SVM are as under.

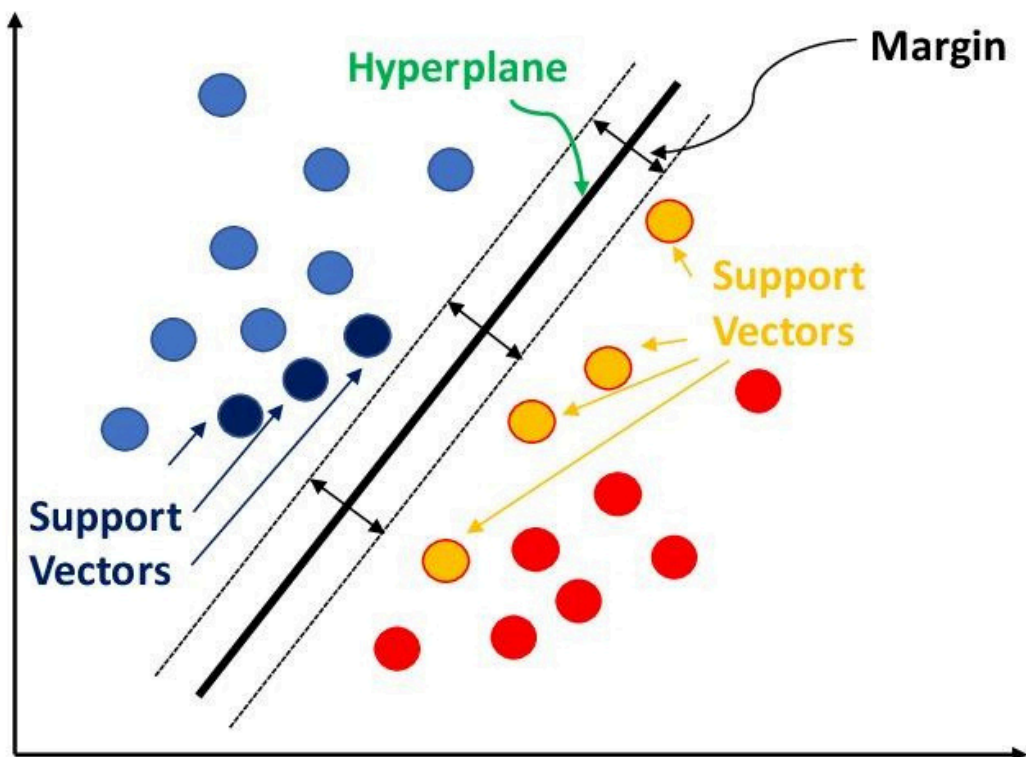
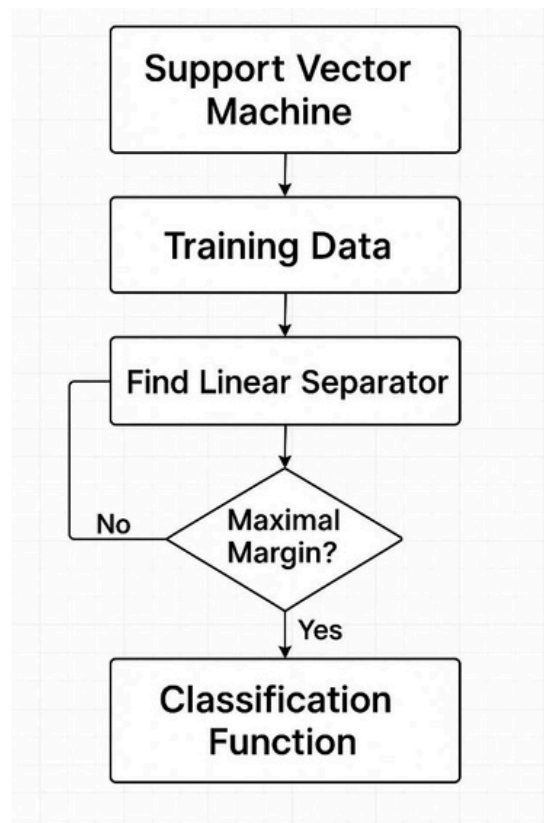
- ✓ **Hyperplane:** A decision boundary that separates different classes.
- ✓ **Margin:** The distance between the hyperplane and the closest data point (support vector) from any class.
- ✓ **Support Vectors:** Data points that lie closest to the hyperplane and influence its position.
- ✓ **Kernel Trick:** Transforms data into a higher-dimensional space to make it linearly separable.

Imagine drawing a line on a field to separate apples from oranges, placing it in such a way that it's as far away as possible from the closest fruit of each type. That's what SVM does finds the safest margin between two classes to avoid confusion, even if the boundary isn't straight.

Types of SVM

- ✓ **LinearSVM** : Used when data is linearly separable.
- ✓ **Non-linearSVM** : Uses kernel functions (e.g., polynomial, RBF) when the data cannot be separated with a straight line.

SVM can be illustrated by the following flowchart



Pseudocode for SVM

Algorithm: SVM_Train(D, kernel)

Input: Training dataset $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$, kernel function K

Output: Support Vectors, hyperplane parameters

1. Map input data to higher-dimensional space (if non-linear)
2. Set up and solve the quadratic optimization problem:
Maximize margin = $1 / \|w\|$ subject to:
 $y_i(w \cdot x_i + b) \geq 1$ for all i
3. Identify support vectors (where constraint is active)
4. Construct decision function:
 $f(x) = \text{sign}(w \cdot x + b)$ or $f(x) = \text{sign}(\sum \alpha_i y_i K(x_i, x) + b)$

Implementation on Python

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Load dataset
iris = load_iris()
X =pd.DataFrame(iris.data, columns=iris.feature_names)
y =pd.Series(iris.target)

# Filter to binary classification
X = X[y != 2]
y = y[y != 2]

# Use only two features for 2D visualization
X =X.iloc[:, [0,1]] # sepal length and sepal width

# Split dataset
X_train,X_test,y_train, y_test = train_test_split(X, y,
test_size=0.3,random_state=42)

# Train SVM (linear kernel)
clf = SVC(kernel='linear')
clf.fit(X_train, y_train)

# Predict and evaluate
y_pred=clf.predict(X_test)

# Colored Confusion Matrix
cm=confusion_matrix(y_test, y_pred)
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='coolwarm',
xticklabels=['Setosa', 'Versicolor'], yticklabels=['Setosa',
'Versicolor'])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix (SVM)")
plt.show()
```

```

# Classification Report
print("\nClassification Report:\n",classification_report(y_test,
y_pred,target_names=['Setosa','Versicolor']))

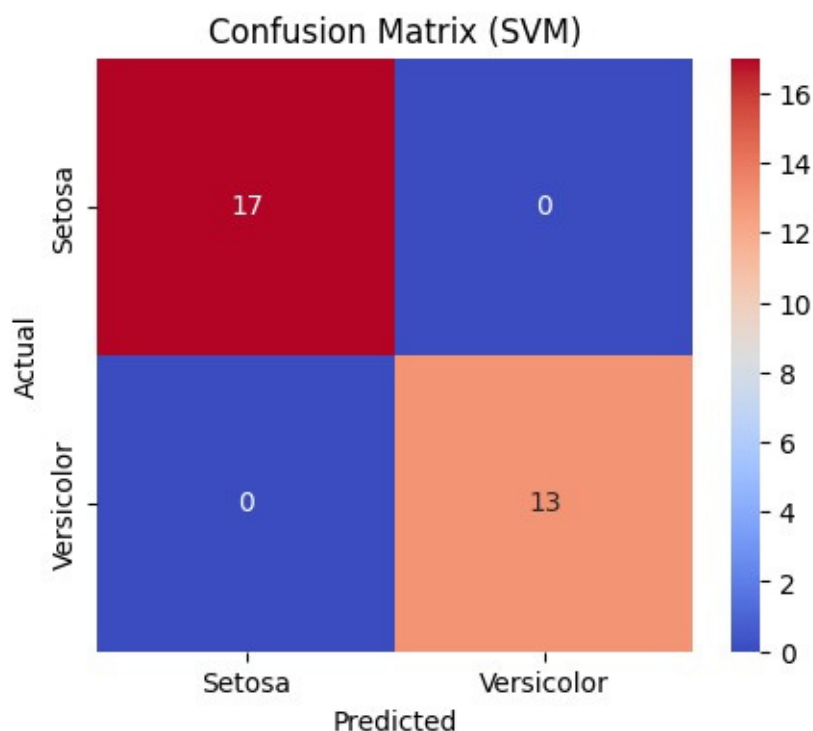
# Decision Boundary Plot
def plot_decision_boundary(X, y, model):
    x_min,x_max=X.iloc[:,0].min()-1,X.iloc[:,0].max()+1
    y_min,y_max=X.iloc[:,1].min()-1,X.iloc[:,1].max()+1
    xx,yy=np.meshgrid(np.linspace(x_min,x_max, 300),
                      np.linspace(y_min,y_max, 300))
    Z=model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

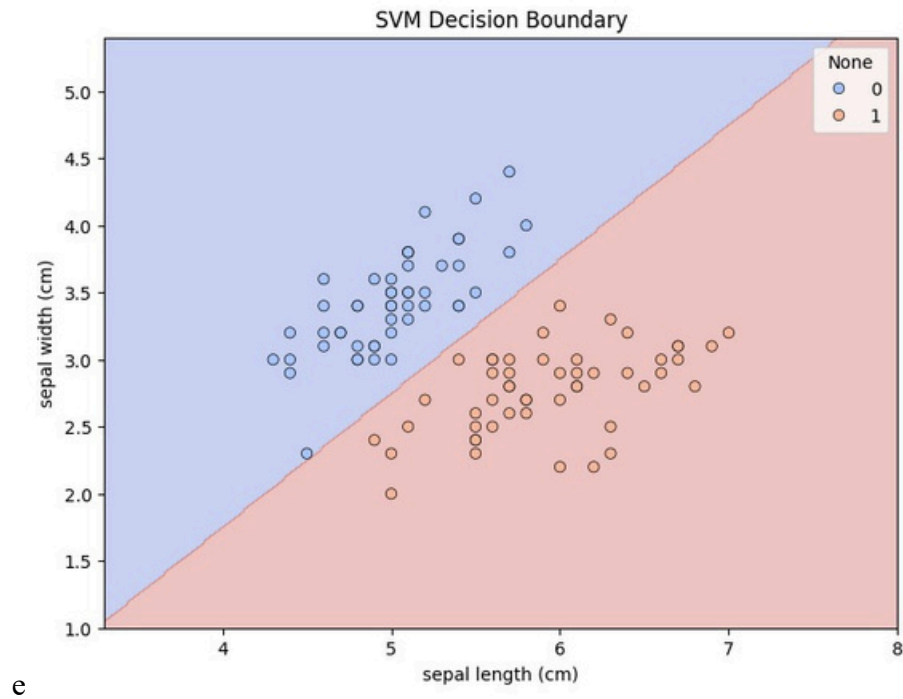
    plt.figure(figsize=(8, 6))
    plt.contourf(xx, yy,Z,alpha=0.3,cmap='coolwarm')
    sns.scatterplot(x=X.iloc[:,0],y=X.iloc[:,1], hue=y,
palette='coolwarm', edgecolor='k')
    plt.xlabel(X.columns[0])
    plt.ylabel(X.columns[1])
    plt.title("SVM DecisionBoundary")
    plt.show()

plot_decision_boundary(X, y, clf)

```

OUTPUT





Performance Comparison of the three classifiers

Here, we will use a real-time problem dataset for the detection of heart disease in the patient and apply the three classifiers on the same dataset. The dataset is nonlinear and noisy and the purpose of the application of the classifiers is to check the accuracy of each in solving a problem. The dataset is available at

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

# Load the heart disease dataset from UCI
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/heart-
disease/processed.cleveland.data"
column_names = [
    'age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg',
    'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target'
]

try:
    # Try to load the dataset from UCI
    df=pd.read_csv(url,names=column_names,na_values='?')

    #Handle missing values-drop rows with missing values for
    simplicity
    df = df.dropna()

    #Convert target to binary(0=nodisease,1-4=disease)
    df['target'] = (df['target'] > 0).astype(int)

except Exception as e:
    print(f"Error loading dataset from UCI: {e}")
    print("Using local backup dataset...")
    #Fallback to a local version if the URL doesn't work
    from sklearn.datasets import load_breast_cancer
    data = load_breast_cancer()
    df=pd.DataFrame(data.data,columns=data.feature_names)
    df['target'] = data.target

# Prepare features and target
X = df.drop(columns=['target'])
y = df['target']

# Split the data
X_train,X_test,y_train,y_test=train_test_split(X,y, test_size=0.3,
random_state=42)

```



```

# Standardize the data
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Initialize classifiers
classifiers = {
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'Random Forest': RandomForestClassifier(random_state=42),
    'SVM': SVC(random_state=42)
}

# Create subplots for confusion matrices
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
plt.suptitle('Confusion Matrices for Different Classifiers', y=1.05,
fontsize=16)

results = []
for i, (name, clf) in enumerate(classifiers.items()):
    # Train and predict
    clf.fit(X_train_scaled, y_train)
    y_pred = clf.predict(X_test_scaled)
    acc = accuracy_score(y_test, y_pred)

    # Store results
    results.append({'Classifier': name, 'Accuracy': acc})

    # Create confusion matrix
    cm = confusion_matrix(y_test, y_pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[i],
xticklabels=['No Disease', 'Disease'],
yticklabels=['No Disease', 'Disease'])
    axes[i].set_title(f'{name}\nAccuracy: {acc:.3f}')
    axes[i].set_xlabel('Predicted')
    axes[i].set_ylabel('Actual')

    # Print classification report
    print(f"\n{name} Classification Report:")
    print(classification_report(y_test, y_pred))

plt.tight_layout()
plt.show()

# Convert results to DataFrame
results_df = pd.DataFrame(results)

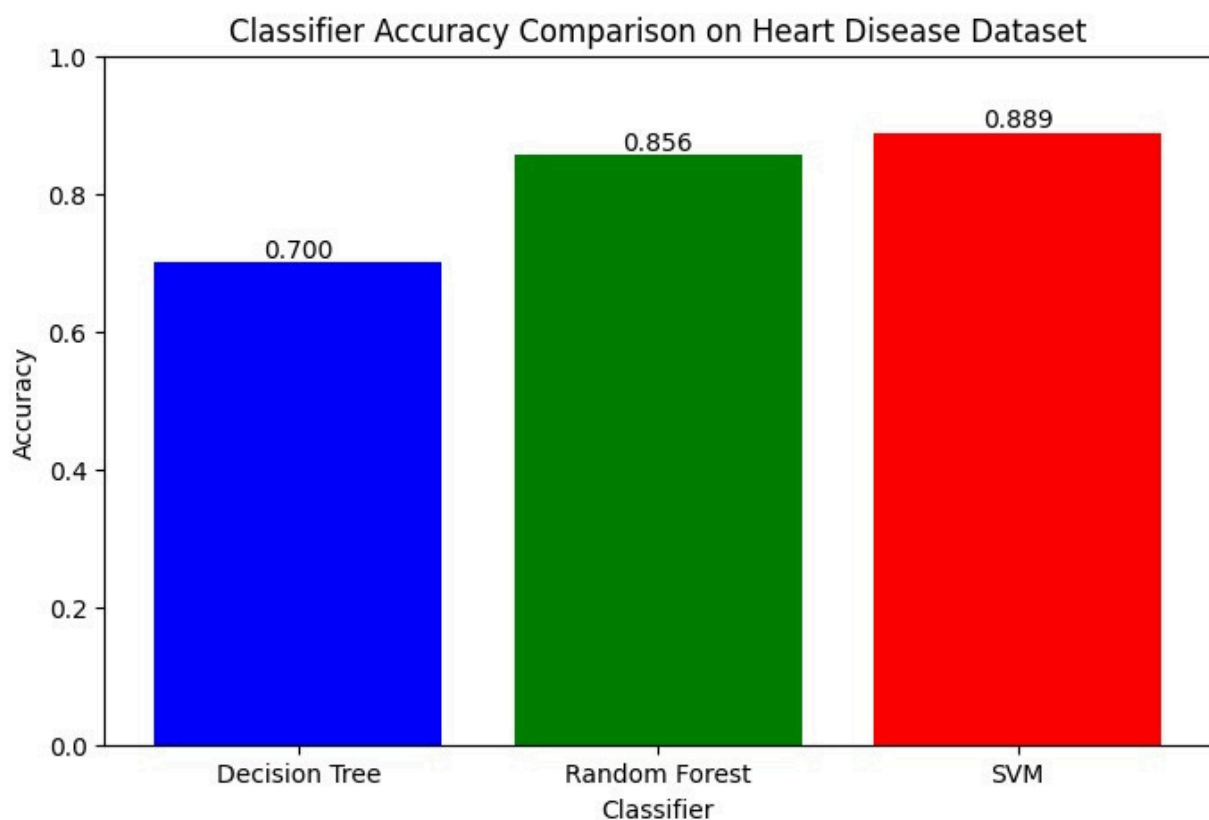
# Display results in table
print("\nClassifier Accuracy Comparison:")

```

```
print(results_df.to_string(index=False))

#plot accuracy comparison
plt.figure(figsize=(8, 5))
bars = plt.bar(results_df['Classifier'], results_df['Accuracy'],
color=['blue', 'green', 'red'])
plt.title('Classifier Accuracy Comparison on Heart Disease Dataset')
plt.xlabel('Classifier')
plt.ylabel('Accuracy')
plt.ylim(0, 1)
for bar in bars:
    height=bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2., height,
             f'{height:.3f}', ha='center', va='bottom')
plt.show()
```

Output



LAB TASKS

1. SVM Classifier Implementation

- ✓ Use a nonlinear kernel (e.g., RBF) on a real-world dataset (e.g., handwriting digits).
- ✓ Visualize support vectors and decision boundaries.

2. Model Comparison Dashboard

- ✓ Create a comparative dashboard using matplotlib or Streamlit to show performance metrics across Decision Tree, RF, and SVM.

3. Feature Importance Visualization

- ✓ For RF, visualize feature importance and interpret which features contribute most to classification.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, classification_report,
accuracy_score
# --- Task 1: SVM Classifier Implementation (Visualize Support Vectors and
Decision Boundaries) ---
# Using Iris dataset for a clear 2D visualization of decision boundaries and
support vectors
print("="*80)

print("TASK 1: SVM Visualization (2D Decision Boundary & Support Vectors)")
print("="*80)

iris = load_iris()
X_2d = iris.data[:, :2] # Using only the first two features for 2D visualization
y_2d = iris.target

# Filter to binary classification for simpler visualization of boundary (e.g.,
setosa vs versicolor)
X_2d_binary = X_2d[y_2d != 2]
y_2d_binary = y_2d[y_2d != 2]

# Split data
```

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, classification_report,
accuracy_score
# --- Task 1: SVM Classifier Implementation (Visualize Support Vectors and
# Decision Boundaries) ---
# Using Iris dataset for a clear 2D visualization of decision boundaries and
# support vectors
print("="*80)

print("TASK 1: SVM Visualization (2D Decision Boundary & Support Vectors)")
print("="*80)

iris = load_iris()
X_2d = iris.data[:, :2] # Using only the first two features for 2D visualization
y_2d = iris.target

# Filter to binary classification for simpler visualization of boundary (e.g.,
# setosa vs versicolor)
X_2d_binary = X_2d[y_2d != 2]
y_2d_binary = y_2d[y_2d != 2]

# Split data
```

```

X_train_2d, X_test_2d, y_train_2d, y_test_2d = train_test_split(X_2d_binary,
y_2d_binary, test_size=0.3, random_state=42)

# Standardize the features
scaler_2d = StandardScaler()
X_train_scaled_2d = scaler_2d.fit_transform(X_train_2d)
X_test_scaled_2d = scaler_2d.transform(X_test_2d)

# Train SVM with RBF kernel
svm_rbf_2d = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)
svm_rbf_2d.fit(X_train_scaled_2d, y_train_2d)

# Create a mesh to plot in
x_min, x_max = X_2d_binary[:, 0].min() - 1, X_2d_binary[:, 0].max() + 1
y_min, y_max = X_2d_binary[:, 1].min() - 1, X_2d_binary[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
np.arange(y_min, y_max, 0.02))

# Plot the decision boundary
plt.figure(figsize=(10, 7))
Z = svm_rbf_2d.predict(scaler_2d.transform(np.c_[xx.ravel(), yy.ravel()]))
Z = Z.reshape(xx.shape)
plt.contourf(xx, yy, Z, cmap=plt.cm.coolwarm, alpha=0.8)

# Plot also the training points
plt.scatter(X_train_2d[:, 0], X_train_2d[:, 1], c=y_train_2d,
cmap=plt.cm.coolwarm, edgecolors='k', s=80, label='Training data')

# Plot the support vectors
plt.scatter(scaler_2d.inverse_transform(svm_rbf_2d.support_vectors_)[:, 0],
scaler_2d.inverse_transform(svm_rbf_2d.support_vectors_)[:, 1],
s=200, facecolors='none', edgecolors='green', linewidths=2,
label='Support Vectors')

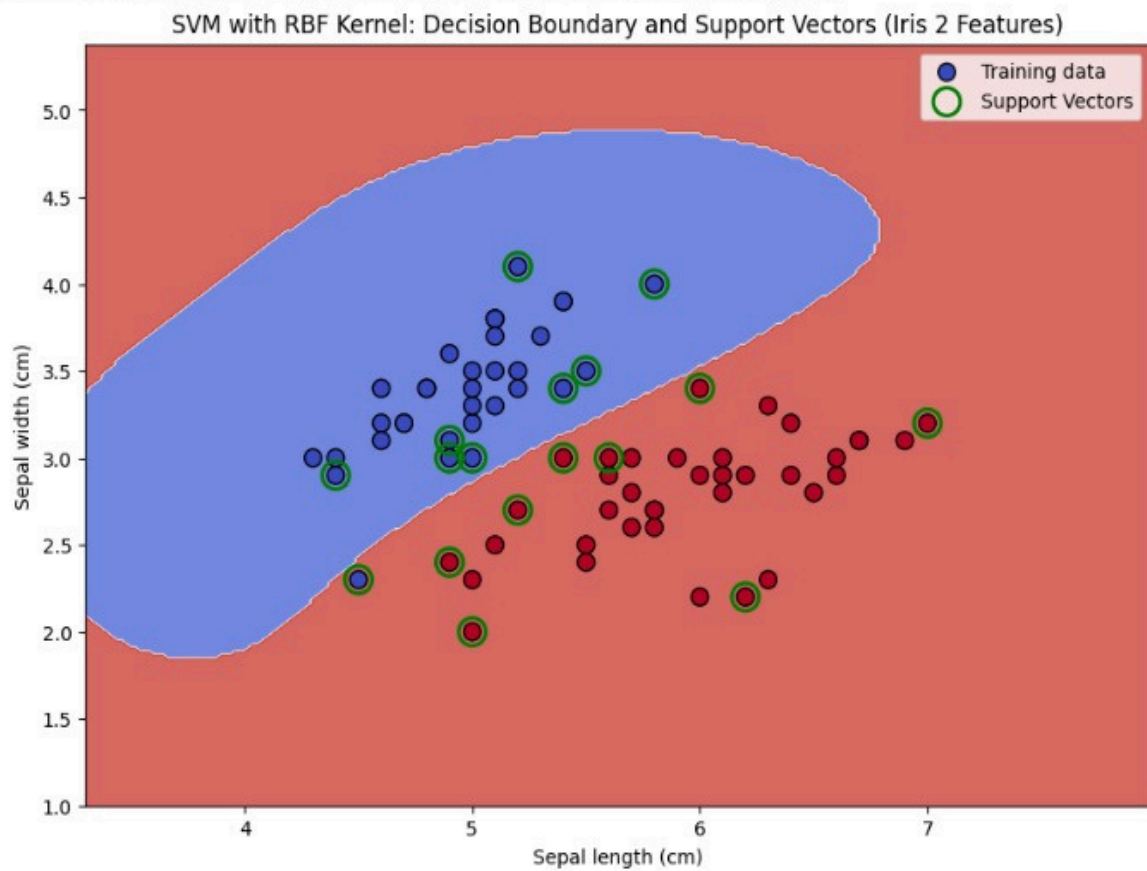
plt.xlabel('Sepal length (cm)')
plt.ylabel('Sepal width (cm)')

```

```
plt.xlim(xx.min(), xx.max())
plt.ylim(yy.min(), yy.max())

plt.title('SVM with RBF Kernel: Decision Boundary and Support Vectors (Iris2
Features)')
plt.legend()
plt.show()
```

```
=====
TASK 1: SVM Visualization (2D Decision Boundary & Support Vectors)
=====
```



Task#02

Code :

Model Comparison Dashboard (Decision Tree, Random Forest, SVM on Iris) ---

```
print("\n"+"="*80)
```

```

print("TASK 2: Model Comparison Dashboard (Iris Dataset)")
print("="*80)

# Load and prepare the Iris dataset (all features)
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
# Standardize the data (important for SVM and can help others)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# Initialize classifiers
classifiers = {
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
    'SVM (RBF)': SVC(kernel='rbf', gamma='scale', random_state=42)
}

results = []

# Create subplots for confusion matrices
fig, axes = plt.subplots(1, 3, figsize=(20, 6))
plt.suptitle('Confusion Matrices for Different Classifiers on Iris Dataset',
y=1.05, fontsize=16)
for i, (name, clf) in enumerate(classifiers.items()):
    print(f"\n-- Training and Evaluating {name} ---")
    # Train and predict

```



```

if name == 'Decision Tree' or name == 'Random Forest':
    clf.fit(X_train, y_train) # DT and RF don't strictly require scaling
    y_pred = clf.predict(X_test)
else: # SVM
    clf.fit(X_train_scaled, y_train)
    y_pred = clf.predict(X_test_scaled)
acc = accuracy_score(y_test, y_pred)
results.append({'Classifier': name, 'Accuracy': acc})
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[i],
            xticklabels=iris.target_names,
            yticklabels=iris.target_names)
axes[i].set_title(f'{name}\nAccuracy: {acc:.3f}')
axes[i].set_xlabel('Predicted Label')
axes[i].set_ylabel('True Label')
# Classification Report
print(f"\n{name} Classification Report:")
print(classification_report(y_test, y_pred, target_names=iris.target_names))

plt.tight_layout(rect=[0, 0.03, 1, 0.95]) # Adjust layout to prevent title
overlap
plt.show()
# Display accuracy comparison
results_df = pd.DataFrame(results)
print("\n"+"="*80)
print("Classifier Accuracy Comparison:")
print("="*80)
print(results_df.to_string(index=False))
plt.figure(figsize=(8, 5))

```

```
sns.barplot(x='Classifier', y='Accuracy', data=results_df, palette='viridis')
plt.title('Classifier Accuracy Comparison')
plt.ylim(0.8, 1.0) # Set reasonable y-axis limits for accuracy
plt.ylabel('Accuracy')
plt.xlabel('Classifier')
plt.tight_layout()
plt.show()
```

Task#03

Code:

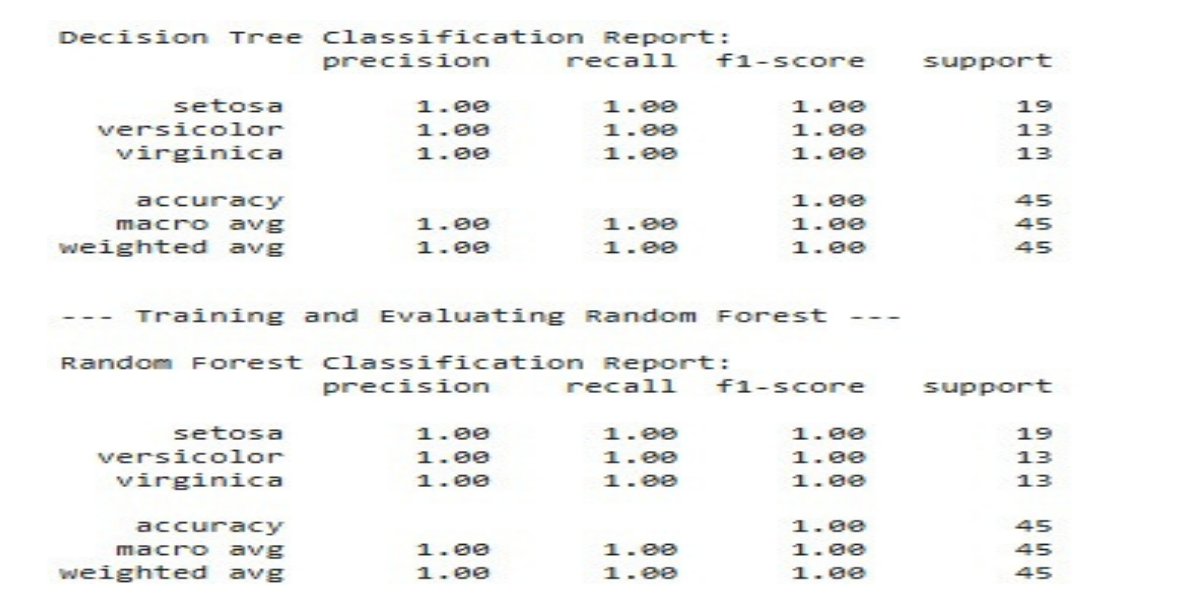
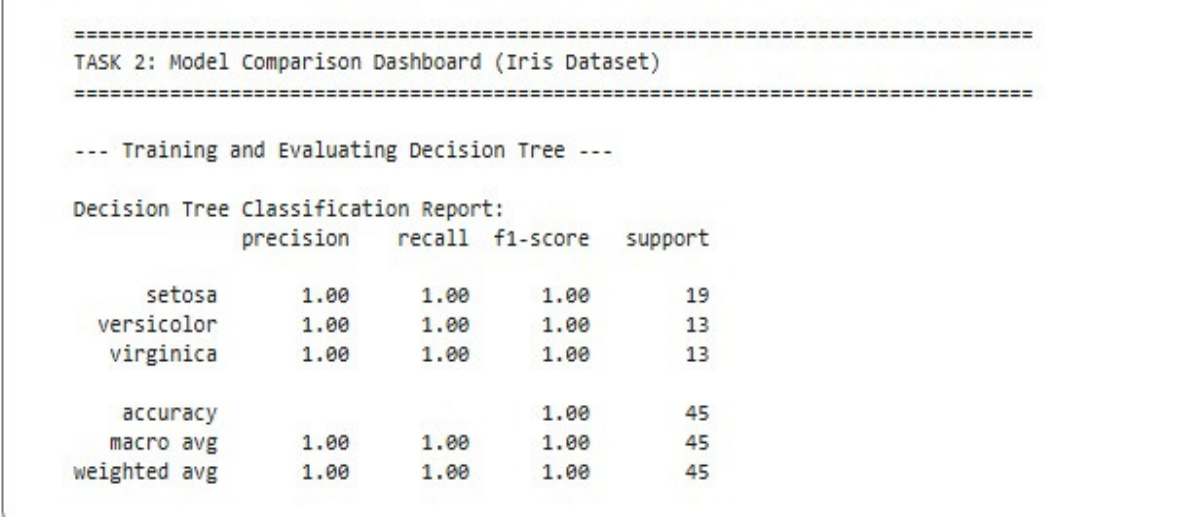
```
print("\n"+"="*80)
print("TASK 3: Random Forest Feature Importance (Iris Dataset)")
print("="*80)

# Re-using the Random Forest model trained in Task 2
rf_model = classifiers['Random Forest']
importances = rf_model.feature_importances_
features = X.columns

# Create a DataFrame for better plotting
feature_importance_df = pd.DataFrame({'Feature': features, 'Importance':
importances})
feature_importance_df = feature_importance_df.sort_values(by='Importance',
ascending=False)
plt.figure(figsize=(10, 6))

sns.barplot(x='Importance', y='Feature', data=feature_importance_df,
palette='magma')
plt.title('Feature Importance in Random Forest (Iris Dataset)')
plt.xlabel('Importance Score')
plt.ylabel('Features')
```

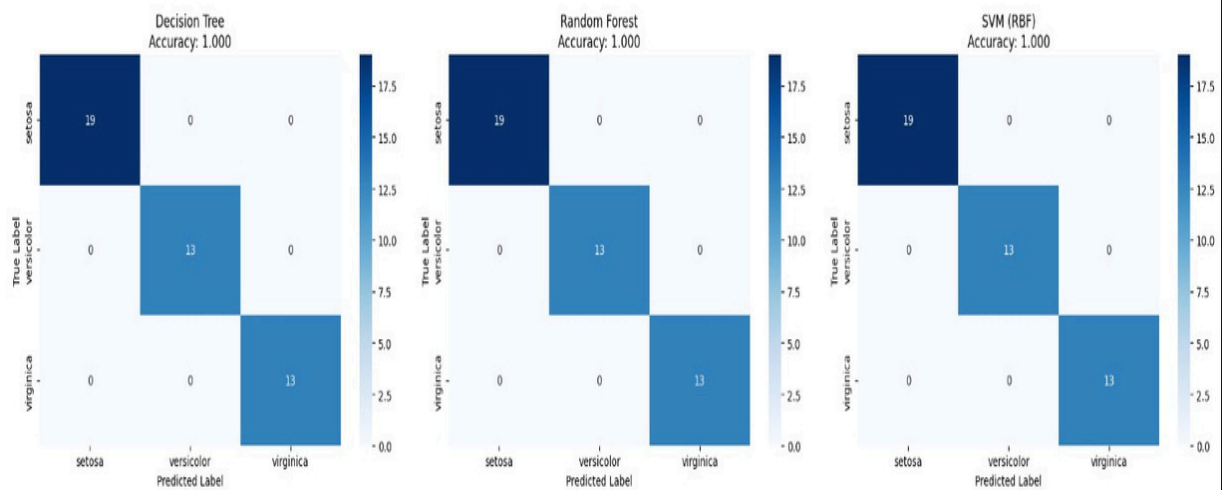
```
plt.tight_layout()
plt.show()
```



--- Training and Evaluating SVM (RBF) ---

SVM (RBF) Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	19
versicolor	1.00	1.00	1.00	13
virginica	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45



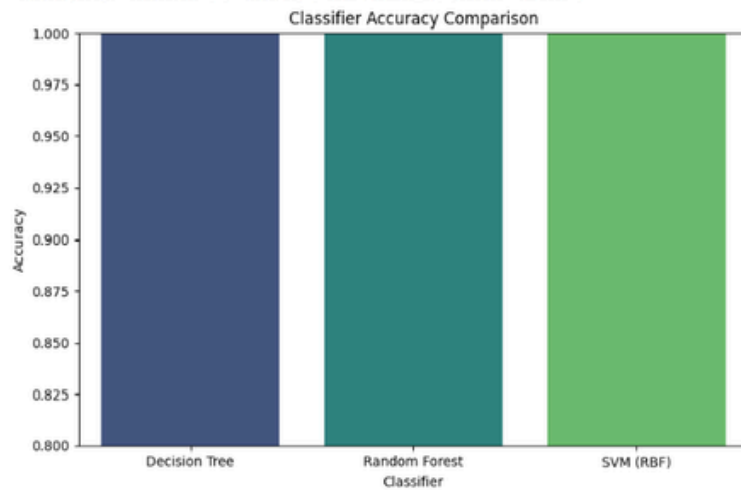
```
=====
Classifier Accuracy Comparison:
=====
```

```
Classifier Accuracy
Decision Tree      1.0
Random Forest     1.0
SVM (RBF)         1.0
```

```
/tmp/ipykernel_2853/627478560.py:134: FutureWarning:
```

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.

```
sns.barplot(x='Classifier', y='Accuracy', data=results_df, palette='viridis')
```

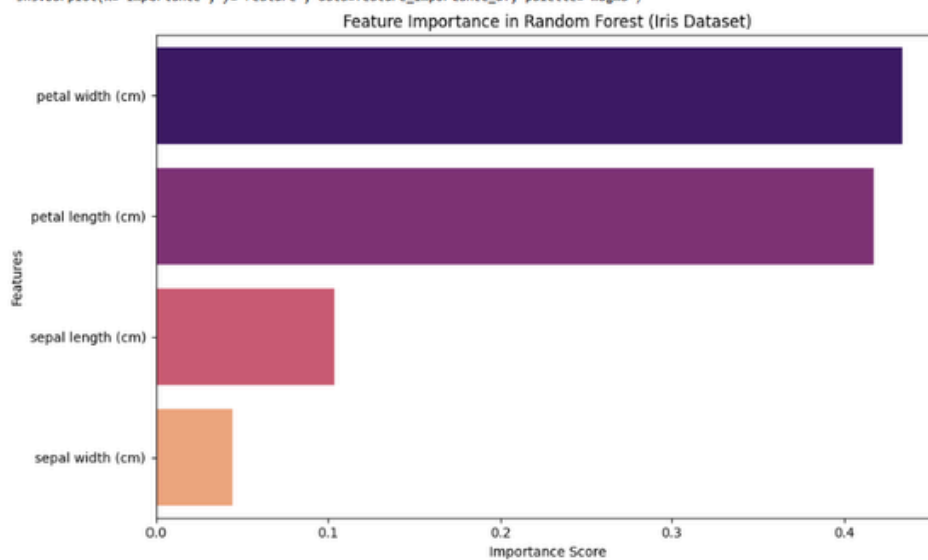


```
=====
TASK 3: Random Forest Feature Importance (Iris Dataset)
=====
```

```
/tmp/ipykernel_2853/627478560.py:157: FutureWarning:
```

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'y' variable to 'hue' and set 'legend=False' for the same effect.

```
sns.barplot(x='Importance', y='Feature', data=feature_importance_df, palette='magma')
```



International Islamic University Islamabad
Faculty of Engineering and Technology
Department of Electrical Engineering

Artificial Intelligence & Machine Learning LAB

Lab No. 11

Advanced Classification Using Decision Tree, Random Forest, and SVM

Name: Bushra Nazir Reg. No.: 1071-F22

S. No.	Criteria	Allocated Percentage	Level 1 (0%)	Level 2 (25%)	Level 3 (50%)	Level 4 (75%)	Level 5 (100%)	Marks Obtained
(A) PSYCHOMOTOR (To be judged in the lab during experiment)								
1	Practical Implementation OR Connectivity/Arrangement of Equipment	5	Absent	Degree of errors in applying procedural knowledge or Degree of errors in arrangement of equipment:				
			0	With several critical errors, Imcomplete and Not Neat	With few errors, Incomplete and not Neat	With some errors, Complete but not Neat	Without errors, Complete and Neat	
				1.25	2.5	3.75	5	
2	Use of Equipment OR Use of Simulation/Programming Tool	2	Absent	Use of tools, equipment and materials with:				
			0	Limited Competence	Some Competence	Considerable Competence	Good Competence	
				0.5	1	1.5	2	
3	Safety (Optional)	0	Absent	Degree of reminders to follow safety procedure with:				
			0	Rarely Follow Safety Rules	Needs Reminders	Needs minor Reminders	Follows Safety Rules	
				0	0	0	0	
SUB-TOTAL (PT)		7	SUB-TOTAL MARKS OBTAINED (PO)					
(B) COGNITIVE (To be judged on the copy of experiment submitted)								
4	Data Record, Analysis and Evaluation OR Algorithm Design	1	Absent	Data record, Analysis and Evaluation or Designed Algorithm is:				
			0	Incorrect /Incomplete	Complete with some errors	Complete with few errors	Complete and Accurate	
				0.25	0.5	0.75	1	
5	Procedural Awareness (Optional)	0	Absent	Procedural awareness of the completed task is:				
			0	Inappropriate	Appropriate with some errors	Appropriate with few errors	Appropriate	
				0	0	0	0	
SUB-TOTAL (CT)		1	SUB-TOTAL MARKS OBTAINED (CO)					
(C) AFFECTIVE (To be judged in the lab during experiment)								
6	Level of Participation & Attitude to Achieve Individual/Group Goals	2	Absent	Degree of positive interaction as an individual or within a group:				
			0	Rare interaction	Some interaction	Acceptable interaction	Good interaction	
				0.5	1	1.5	2	
SUB-TOTAL (AT)		2	SUB-TOTAL MARKS OBTAINED (AO)					

Total Marks (PT + CT + AT) : 10

Marks Obtained (PO + CO + AO) :

Instructor